COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	9/9/26	**DATE SUBMITTED:**	9/16/26
NAME:	Estorco, CLint Hoven A.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 **Sorting Algorithms Pt. 2**

1. PROCEDURE OUTPUT

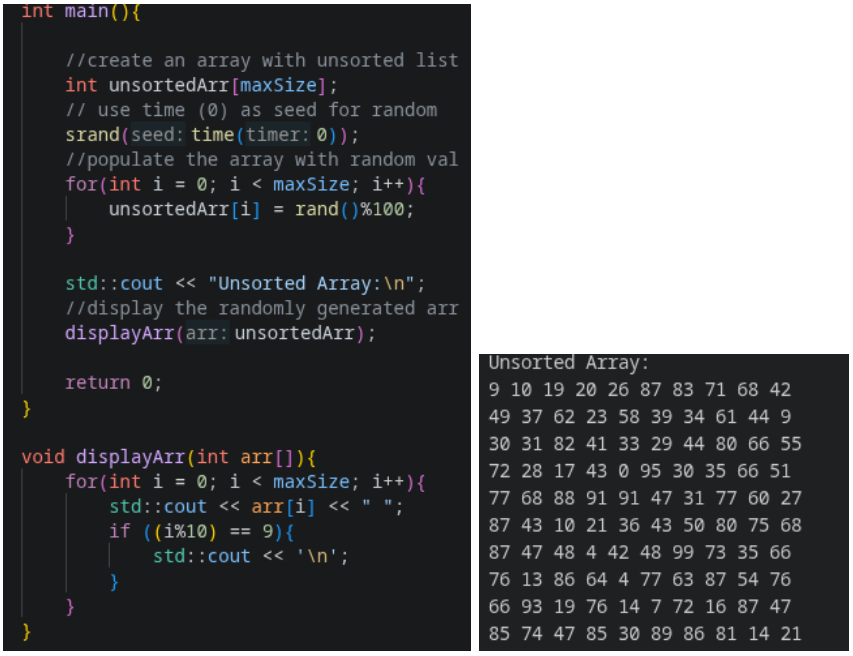
Code + Console Screenshot	
Observations	This code seeds the array of size n with pseudo-random numbers from rand() using the seed from time(0). It then calls displayArr() to print out the values of the array, printing each value in a new line. The overall run time complexity is O(n) for the best case as well as worst case.

Table 8-1. Array of Values for Sort Algorithm Testing

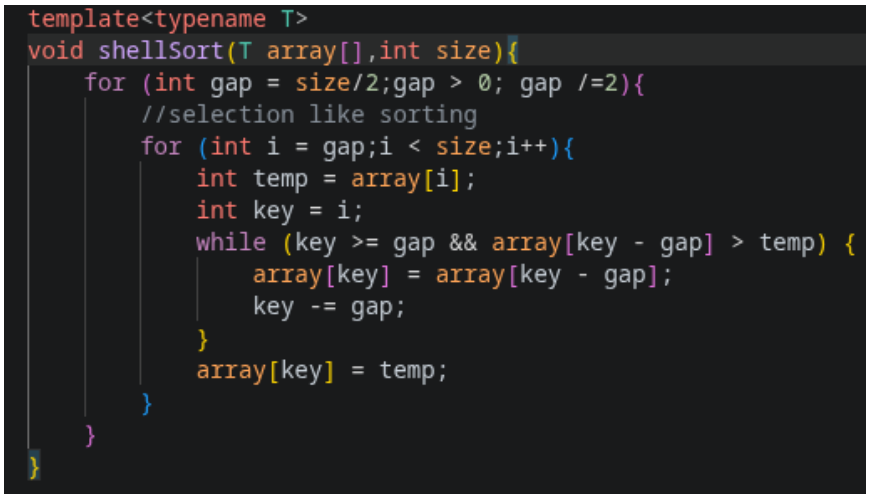
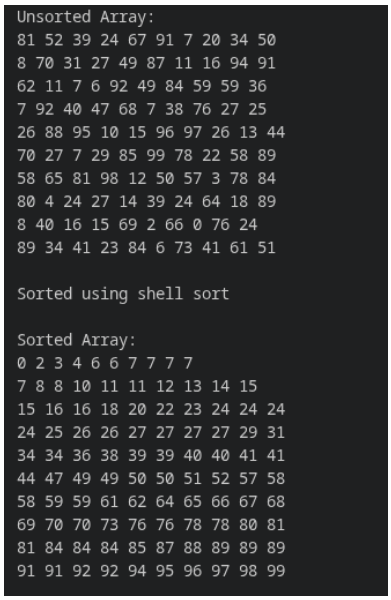
Code + Console Screenshot	 
Observations	The algorithm uses a contraction of the gap, as the gap reduces around the common images of displacement are minimized. The gap keeps progressively decreasing throughout iterations until stabilized on a last iteration - followed by a standard insertion pass. If the displacement sequence used is a standard halving gap sequence then the best-case performance is O(n log n) and the worst case performance is O(n^2).

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```cpp
template<typename T>
void merge(T array[],int left,int mid,int right){
    int sizeL = mid-left + 1;
    int sizeR = right - mid;

    T arrL[sizeL];
    T arrR[sizeR];

    for (int i = 0; i < sizeL; i++){
        arrL[i] = array[left + i];
    }

    for (int j = 0; j < sizeR; j++){
        arrR[j] = array[mid + 1 + j];
    }

    int i = 0, j = 0;
    int k = left;

    // Merge the temp arrays back
    // into arr[left..right]
    while (i < sizeL && j < sizeR) {
        if (arrL[i] <= arrR[j]) {
            array[k] = arrL[i];
            i++;
        }
        else {
            array[k] = arrR[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L[],
    // if there are any
    while (i < sizeL) {
        array[k] = arrL[i];
        i++;
        k++;
    }

    // Copy the remaining elements of R[],
    // if there are any
    while (j < sizeR) {
        array[k] = arrR[j];
        j++;
        k++;
    }

}
```

```cpp
template<typename T>
void mergeSort(T array[],int left,int right){
    if (left >= right)return;
    int mid = left + (right - left) / 2;
    mergeSort(array, left, right: mid);
    mergeSort(array, left: mid + 1, right);
    merge(array, left, mid, right);
}
```

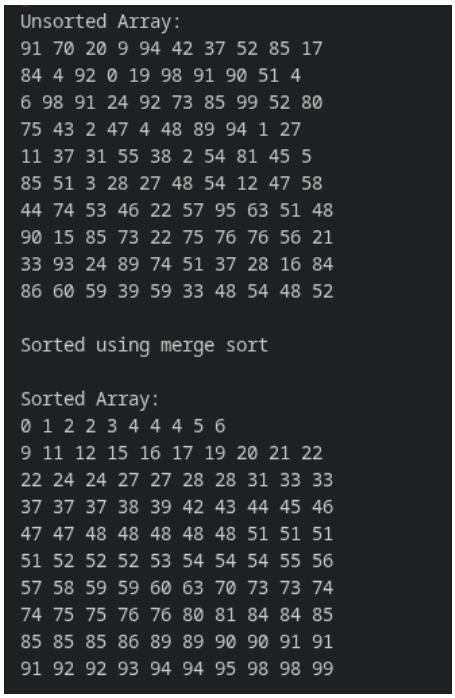
	
Observations	It is done by recursive bisection where it partitions the array into sub-arrays repeatedly, and eventually combines them into a sorted array. Tiny pieces go on first in reconstruction so that they can be used to make sure things line up. First the algorithmic decomposition makes the operation a logarithmic order (log n) and the second one the merge step (n) makes the total time average order (n log n).

Table 8-3. Merge Sort Algorithm

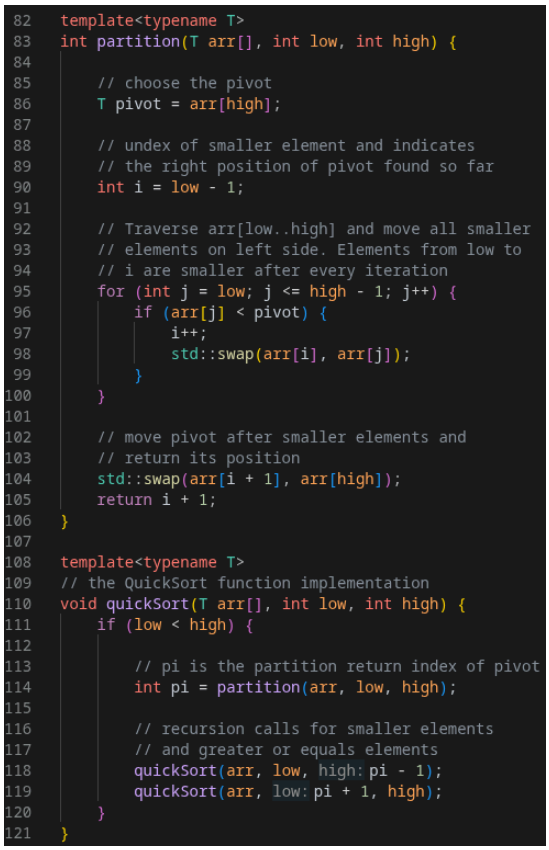
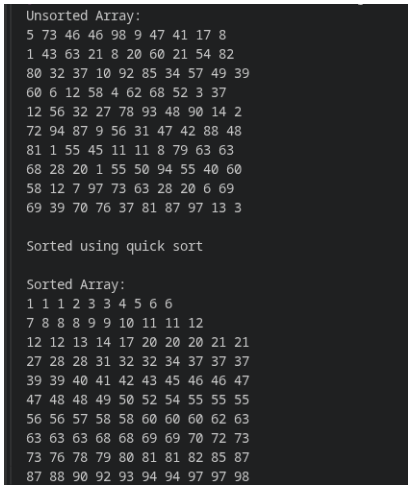
Code + Console Screenshot	 
Observations	This divide and conquer algorithm also partitions, by designating a pivot value, so that all the values on the left side are smaller than it and all values on the right side are larger. It orients itself towards the base cases (zero and one) by repeatedly recursively performing the above procedure on smaller sub-arrays. The runtime is O(n \log n) when the pivot selections create a nearly balanced partition, and it is O(n^2) when the partitions are very unbalanced.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
template <typename T>
void shellSort(T arr[], int size) {
    for (int interval = size / 2; interval > 0; interval /= 2) {
        for (int i = interval; i < size; i++) {
            T temp = arr[i];
            int j;
            for (j = i; j >= interval && arr[j - interval] > temp; j -= interval) {
                arr[j] = arr[j - interval];
            }
            arr[j] = temp;
        }
    }
}
```

```
Original Unsorted Dataset:
19 68 44 27 74 46 54 36 93 16 8 76 83 0 95 56 35 94 85 52 72 31 12 36 60 30 70 50 22 51 39 95 28 1 7 10 10
11 74 48 35 87 2 32 4 58 16 19 18 10 85 40 15 60 54 8 17 12 65 24 50 35 98 57 99 93 71 9 60 99 96 52 8 39 9
7 74 14 45 88 43 82 88 50 47 0 89 9 67 65 87 71 87 4 40 45 98 16 79 46 75

Shell Sort Result:
0 0 1 2 4 4 7 8 8 8 9 9 10 10 10 11 12 12 14 15 16 16 16 17 18 19 19 22 24 27 28 30 31 32 35 35 35 36 36 39
 39 40 40 43 44 45 45 46 46 47 48 50 50 50 51 52 52 54 54 56 57 58 60 60 60 65 65 67 68 70 71 71 72 74 74 7
4 75 76 79 82 83 85 85 87 87 87 88 88 89 93 93 94 95 95 96 97 98 98 99 99
```

```
Problem 1: Hybrid Quick Sort
Before Hybrid Quick Sort: 48 12 87 34 4 53 29

After Hybrid Quick Sort: 4 12 29 34 48 53 87
```

Figure 1: Partition using different sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.
Yes, An array can be split into two subarrays, with each component being sorted separately, and then merged.After being split into two subarrays, these can be sorted separately and joined together with another sort. As seen above, invoked steps of secondary sorting were made after partitioning, instead of the normal quicksort (). Even with this change, the "hybrid" algorithm is still able to sort the desired array efficiently with the

same expected time complexity of O(n \log n).

Main (4).cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int SIZE = 100;

void printArray(const int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << "\n\n";
}

void copyArray(const int source[], int destination[], int size) {
    for (int i = 0; i < size; i++) {
        destination[i] = source[i];
    }
}

int main() {
    int originalDataset[SIZE];
    int testArray[SIZE];
```

Main (4).cpp > ...

```
23      int testArray[SIZE];
24
25      srand(time(0));
26      std::cout << "Original Unsorted Dataset: \n";
27      for (int i = 0; i < SIZE; i++) {
28          originalDataset[i] = rand() % 100;
29      }
30      printArray(originalDataset, SIZE);
31
32      copyArray(originalDataset, testArray, SIZE);
33      shellSort(testArray, SIZE);
34      std::cout << "Shell Sort Result:\n";
35      printArray(testArray, SIZE);
36
37      copyArray(originalDataset, testArray, SIZE);
38      mergeSort(testArray, 0, SIZE - 1);
39      std::cout << "Merge Sort Result: \n";
40      printArray(testArray, SIZE);
41
42      copyArray(originalDataset, testArray, SIZE);
43      quickSort(testArray, 0, SIZE - 1);
44      std::cout << "Quick Sort Result: \n";
```

```
44      std::cout << "Quick Sort Result: \n";
45      printArray(testArray, SIZE);
46
47      std::cout << "Problem 1: Hybrid Quick Sort\n";
48
49      int prob1Array[] = {48, 12, 87, 34, 4, 53, 29};
50      int prob1Size = 7;
51
52      std::cout << "Before Hybrid Quick Sort: ";
53      printArray(prob1Array, prob1Size);
54
55      hybridQuickSort(prob1Array, 0, prob1Size - 1);
56
57      std::cout <<"After Hybrid Quick Sort: ";
58      printArray(prob1Array, prob1Size);
59
60      return 0;
61  }
```

```
Merge Sort Result:
0 0 1 2 4 4 7 8 8 8 9 9 10 10 10 11 12 12 14 15 16 16 16 17 18 19 19 22 24 27 28 30 31 32 35 35 35 36 36 39
 39 40 40 43 44 45 45 46 46 47 48 50 50 50 51 52 52 54 54 56 57 58 60 60 60 65 65 67 68 70 71 71 72 74 74 7
4 75 76 79 82 83 85 85 87 87 87 88 88 89 93 93 94 95 95 96 97 98 98 99 99

Quick Sort Result:
0 0 1 2 4 4 7 8 8 8 9 9 10 10 10 11 12 12 14 15 16 16 16 17 18 19 19 22 24 27 28 30 31 32 35 35 35 36 36 39
 39 40 40 43 44 45 45 46 46 47 48 50 50 50 51 52 52 54 54 56 57 58 60 60 60 65 65 67 68 70 71 71 72 74 74 7
4 75 76 79 82 83 85 85 87 87 87 88 88 89 93 93 94 95 95 96 97 98 98 99 99
```

Figure 2: Quick and Merge Sort Comparison

Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?
Merge Sort is not as efficient as Quick Sort for this particular set of data. Quick Sort uses in-place divide and conquer methods while both paradigms attempt to achieve an average time complexity of O(n log n) by

performing logarithmic time for array partitioning (QuickSort) or merging (MergeSort) and linear time to decompose the arrays. This results therefore in a lower memory consumption and smaller constant factors, which lead to significantly faster execution speeds in main memory.

Conclusion:

In this activity, I was able to learn more sorting algorithms. I think that these new algorithms are more sophisticated than the sorting algorithms in the first part. In the following algorithms, recursive calls are used, and other dynamic programming methods. The algorithms can be faster, as they can complete the task with O(n log n) time when compared with the algorithms from the previous activity. The implementation of these algorithms turns out to be more difficult than in the first algorithms, where the mentioned DP is used for the algorithms. I was able to visualize these algorithms from YouTube and other resources to code them. The additional material also enabled investigating the implementations of these algorithms and comparing them with a test case. My goal with the algorithms is that I will develop better skills in implementing them and when to use them.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.